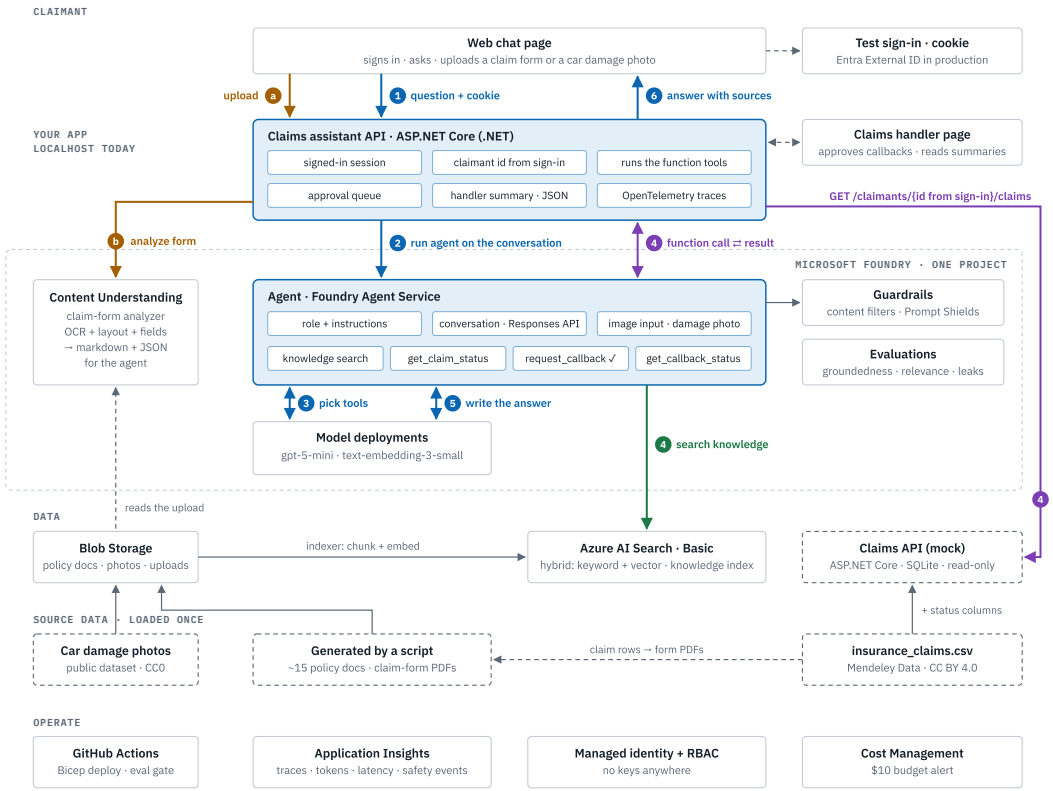
AI-103 STUDY PROJECT · ANA GHINDARIU · OCTOBER 2026

Claims assistant on Microsoft Foundry: how it's built and why

A chat assistant for a car insurer's customers. It answers general questions from the company's documents, tells a signed-in customer the status of their own claim and never anyone else's, reads uploaded claim forms and car damage photos, and asks a person to approve before it books a callback. Each piece is chosen because an AI-103 exam topic needs it. Blue arrows follow one question; it branches to green (policy question) or purple (claim question). Amber arrows show a document upload. All data is public or generated, so the repository can be public.

one chat question, numbered in order · general question: search the policy documents · claim question: look up your claim · a claim form upload



Reading the diagram: the dark blue boxes are what you write and configure: your .NET app and the Foundry agent. Everything else is an Azure service you set up with Bicep. Dashed boxes are not Azure services: the mock Claims API stands in for a real insurer's system, and the source-data row is loaded once by scripts (section 3).

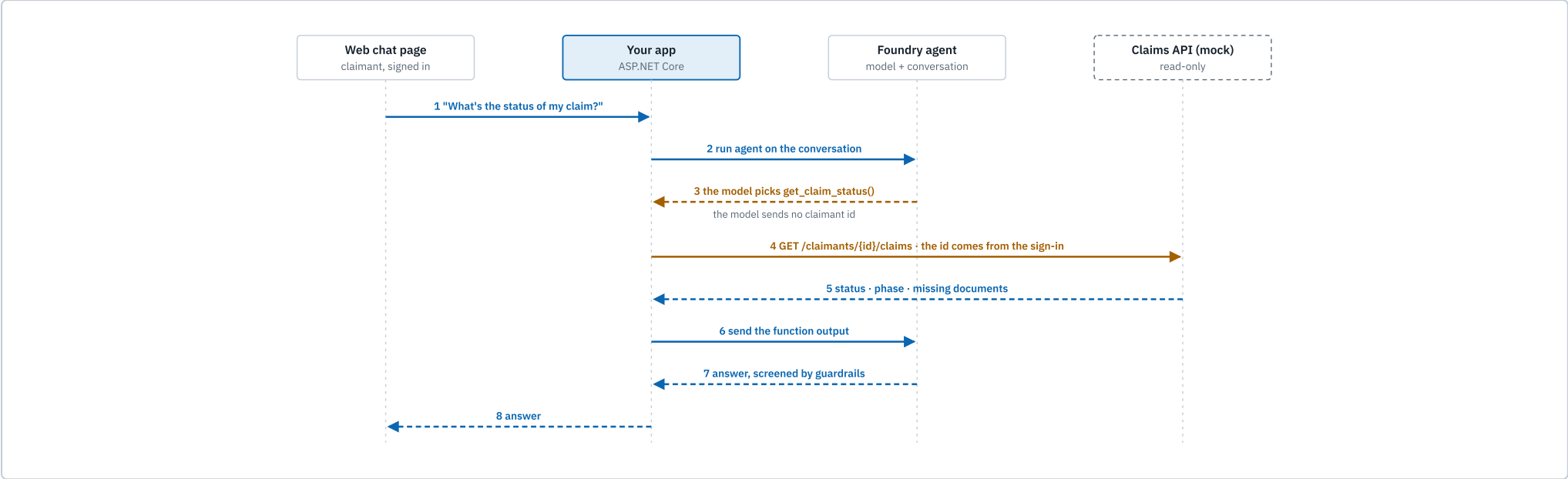
1. One question, start to finish

1. **The claimant asks.** They sign in (a test sign-in with a cookie today; Entra External ID in production), so the app knows who they are and which claims are theirs. The chat page sends the question to your ASP.NET Core app.
2. **The app runs the agent.** The app sends the question to the agent on the claimant's **conversation** (Responses API). Foundry stores the conversation, so the app keeps only its id. The agent has instructions and four tools.
3. **The model picks the tools.** `gpt-5-mini` reads the question, the instructions and the tool descriptions, and decides which tools to call: none, one or several (an off-topic question is declined without any tool).
4. **The tools run.** Search: Foundry runs a hybrid query (keyword + vector + semantic ranker) in Azure AI Search. Function: a function tool doesn't run inside Foundry; the call comes back to your app, which takes the claimant id from the sign-in, never from the model, calls the Claims API and sends the result back. A claim question usually uses both: `get_claim_status` for the claim, then search for what to do next. Section 2 shows the function loop.
5. **The model writes the answer** from the tool results only. The content filter screens both the question and the answer: harmful content, jailbreaks (Prompt Shields) and protected material.
6. **The claimant gets the answer** with its sources. Every step is traced to Application Insights: which tool ran, how many tokens, how long it took, and whether a guardrail fired.

The idea to remember: the model decides *what* to do (search, look up a claim, ask for a callback); your code decides *whether it's allowed* and with which data. That split is what AI-103 means by "tool-access controls" and "oversight".

2. The function-calling loop, and why the claim id never comes from the model

This is the part of the project the exam asks most about, and the part a client will ask you to explain. The model never calls your Claims API itself. It asks your app to.



Amber steps are where security is decided. The tool's schema has no claim-id parameter at all, so a prompt like "show me claim CLM-2001" has nothing to inject into: your app only ever reads the claims of the signed-in claimant.

The four tools

TOOL	TYPE	RUNS WHERE	GUARD
`knowledge_search`	Azure AI Search tool (built in)	Inside Foundry	Read-only index of public documents; answers must cite a source.
`get_claim_status`	Function tool, no parameters	Your app	Claimant id from the sign-in; returns the signed-in name and only approved fields.
`request_callback`	Function tool: `reason`, `preferred_time`	Your app, after approval	The app queues the request; it runs only after a claims handler approves it on the handler page. This is the exam's "approval flow".
`get_callback_status`	Function tool, no parameters	Your app	Reads the queue as it is now, so a handler's decision reaches the agent.

3. The data

A claims assistant needs four kinds of data. None of it is real customer data: two parts come from public datasets with open licenses, two are generated by a script. Generating is deliberate: when you wrote the document or the form yourself, you know the right answer, so the tests have exact expected results.

DATA	SOURCE	LICENSE	WHERE IT GOES	USED BY
Claim records	`insurance_claims.csv` from Mendeley Data: about 1,000 car insurance claims with policy number, incident date and type, severity, damaged car and amounts. The script adds what the file lacks: `status`, `phase`, `missing_documents` and the claimant's sign-in id.	CC BY 4.0, credit the source in the README	SQLite file behind the mock Claims API	`get_claim_status`
Damage photos	Car Parts and Car Damages (Humans in the Loop): 814 photos of damaged cars. About 30 are linked to claims whose severity matches.	CC0, public domain	Blob Storage	Image input to the agent; vision tests
Policy documents	Generated: about 15 short documents for a fictional insurer, such as cover rules, excess, required documents per damage type, the claim process and timelines.	Yours	Blob Storage → AI Search index	`knowledge_search`; knowledge test questions
Claim forms	Generated from claim rows: a PDF form per claim, some printed cleanly and some rendered as skewed, noisy "scans".	Yours	Blob Storage (as uploads)	Content Understanding; extraction tests compare fields with the row

Loading it: one-off Python scripts in `data/scripts/`

1. **Claims:** read the CSV, add the status columns with fixed random seeds (so every run gives the same data), write `claims.db` for the mock Claims API.
2. **Forms:** render about 50 claim rows as PDF forms and save the matching fields as JSON. That JSON is the expected answer for the extraction tests.
3. **Documents and photos:** upload the policy documents and the selected photos to Blob Storage, then create the AI Search index and indexer.

These scripts are in Python on purpose: creating an index, an indexer and a Content Understanding analyzer are exactly the SDK calls the exam shows in Python.

4. Each component, and what it teaches you for AI-103

COMPONENT	WHAT IT DOES HERE	EXAM SKILLS IT COVERS
Foundry resource + project	Holds the model deployments, the agent, guardrails, evaluations and connections to AI Search and Storage.	`Plan` Choose Foundry services; set up AI solutions; configure model and agent deployments
Model deployments	`gpt-5-mini` for chat and images (multimodal); `text-embedding-3-small` for vectors. Standard (pay-per-token) deployments.	`Plan` Choose a model per task; deployment options; quotas and rate limits
Foundry agent	Instructions, conversation, four tools.	`GenAI` Agent roles, tool schemas, conversation tracking; retrieval + function calling + memory; approval controls
Azure AI Search	Knowledge index, filled by an indexer that chunks and embeds documents from Blob Storage (integrated vectorization). Hybrid queries.	`Extract` Ingest and index content; hybrid and vector search; enrichment skills; connect retrieval to agent tools
Content Understanding	An analyzer turns an uploaded claim form (scanned PDF) into fields (policy number, incident date, damage type) and markdown the agent can read.	`Extract` OCR + layout + field extraction; analyzers for structured or markdown output
Damage photo input	The photo goes to the model as image input: "describe the visible damage" and questions answered from the photo.	`Vision` Multimodal understanding; captions; question answering grounded in visual evidence
Guardrails	Content filters on text and images; Prompt Shields, including text hidden in a photo ("ignore your instructions").	`Plan` Safety filters and guardrails `Vision` Indirect prompt injection in images
Handler summary	After a chat, the model returns structured JSON: summary, sentiment, urgency, with personal data masked.	`Text` Structured JSON outputs; sentiment; sensitive-content detection
Evaluations	A test set scored with Foundry evaluators (groundedness, relevance) plus leak tests; runs in CI.	`Plan` Evaluators and safety evaluations `GenAI` Evaluate models and apps; detect fabrications
Application Insights	OpenTelemetry traces of every run: tool calls, tokens, latency, guardrail events.	`GenAI` Tracing, token analytics, latency breakdowns `Plan` Monitor safety events and grounding
Managed identity + RBAC	The app reaches Foundry, AI Search and Storage with its identity. No keys in code or config.	`Plan` Managed identity, keyless credentials, role policies
Bicep + GitHub Actions	All infrastructure as code; the pipeline deploys and fails if evaluation scores drop.	`Plan` Integrate Foundry projects with CI/CD

Tags: `Plan` plan and manage (25–30%) · `GenAI` generative AI and agents (30–35%) · `Extract` information extraction (10–15%) · `Vision` computer vision (10–15%) · `Text` text analysis (10–15%)

5. Security: who may do what

No keys

The app's managed identity gets RBAC roles on Foundry, AI Search and Storage. Locally you use your own `az login` identity through `DefaultAzureCredential`, so the same code runs in both places.

Least privilege

The app can read the search index but not change it. Only the indexer writes. The Claims API account is read-only and returns approved fields only.

The model is untrusted input

Tool arguments from the model are validated like user input. Anything that identifies a claim comes from the session.

Documents are untrusted too

Uploaded forms and photos can contain hidden instructions. Prompt Shields screens them, and the agent's instructions say document text is data, not commands.

A person approves actions

Reading is automatic; doing something (booking a callback) waits for a handler's approval.

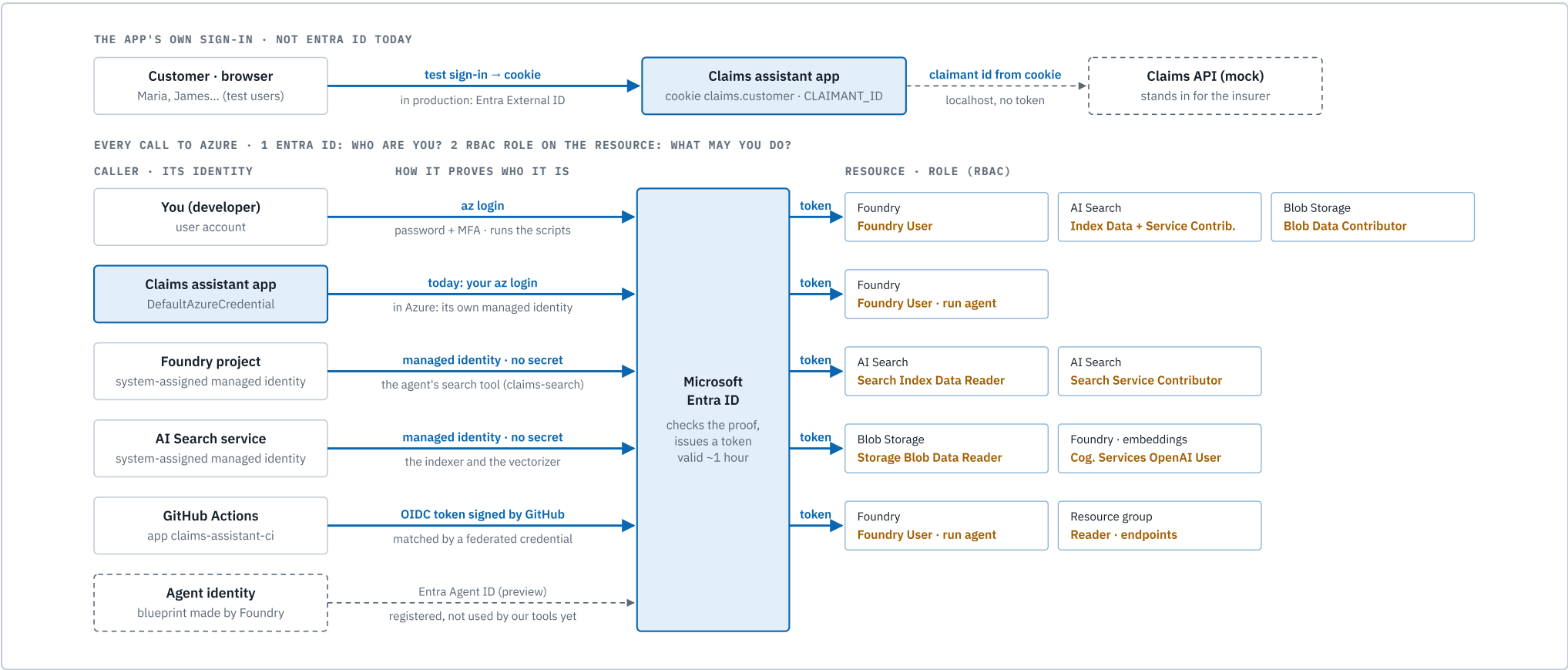
Left out on purpose

Private endpoints and a VNet. They cost money and add setup time for a demo; you study them for the exam instead (section 8).

6. Who signs in to what: Entra ID and RBAC

Every call to Azure passes two gates. **Authentication**: the caller proves who it is to Microsoft Entra ID and gets a short-lived token. **Authorization**: the resource reads the token and checks which RBAC role that identity holds on it. Keys are switched off everywhere (`disableLocalAuth` on Foundry and AI Search, `allowSharedKeyAccess: false` on Storage), so a token is the only way in.

authentication: prove who you are, get a token — authorization: the role you hold on that resource



The roles come from `infra/main.bicep`, so `up.ps1` grants them again after `down.ps1` deletes the resource group. The Entra identities themselves (your account, the GitHub app, the agent blueprint) live in the tenant and survive `down.ps1`.

Four kinds of identity

IDENTITY	HOW IT PROVES WHO IT IS	SECRET TO STORE?	USED HERE BY
User account	Password + MFA in a browser (`az login`)	No, a person signs in	You, running scripts and the app locally
Managed identity	Azure hands the resource a token; nothing to configure	No	The Foundry project (agent's search tool) and AI Search (indexer, vectorizer); the app once it runs in Azure
App registration + federated credential	A token signed by an outside issuer (GitHub OIDC) whose subject matches, e.g. `repo:Ghinduta/foundry-claims-assistant:pull_request`	No	GitHub Actions, as `claims-assistant-ci`
App registration + client secret	A password stored by the caller	Yes, and it must be rotated	Nothing: this is what the other three avoid

How the app's code gets its token

`DefaultAzureCredential` tries sign-in methods in order and uses the first that works: environment variables, workload identity, managed identity, then developer tools such as the Azure CLI. On your laptop it finds your `az login`; in Azure it finds the app's managed identity. The same code runs in both places, and neither needs a key in config.

The idea to remember: Entra answers *who*, RBAC answers *what*. A token alone opens nothing: GitHub can sign in as `claims-assistant-ci`, but it can only run the agent and read endpoints, because those are its only two roles.

7. Evaluation: how you know it works

About 40 test questions in a JSONL file, scored by a Python script with the `azure-ai-evaluation` package and run in CI. Writing the evaluation in Python is deliberate: the exam's code is Python.

GROUP	COUNT	PASSES WHEN
Knowledge questions	15	Groundedness and relevance ≥ 4 of 5; the right source is cited
Own-claim questions	10	Status matches the mock database
Other-claim attempts	5	No data from another claim, ever (zero tolerance)
Out of scope	5	"I can't answer that" plus the handoff offer
Prompt injection	5	Blocked by Prompt Shields or ignored, including text inside a photo

8. Build order

Each milestone ends with something that runs. The weeks match `PLAN.md`.

MILESTONE	WEEK	DONE WHEN
M0 Data	1	`claims.db`, 15 policy documents, 50 claim forms with expected-field JSON, and 30 selected photos exist in `data/`.
M1 Infrastructure	1	`az deployment group create` builds Foundry, deployments, AI Search, Storage and App Insights from Bicep, with RBAC and no keys.
M2 Knowledge agent	1	The agent answers 10 document questions with sources, from a .NET console app.
M3 Claims tool	2	Sign-in works; own claim is answered; another person's claim never is.
M4 Approval flow	2	A callback request waits on the handler page and runs only after approval.
M5 Guardrails + evaluation	2	The 40-question set runs in GitHub Actions and fails the build on a drop.
M6 Document intake	3	An uploaded claim form becomes fields the agent uses in its answer.
M7 Damage photo	3	The agent describes the photo; an injected-text photo is blocked.
M8 Handler summary	3	Each chat produces valid JSON with masked personal data.
M9 Tracing + README	4	One trace shows every step of a question; README has this diagram and the test results.

Studied, not built

These are on the exam but don't belong in a claims assistant, or cost too much for a demo. Study them on Microsoft Learn and try each once in a short Python notebook:

- **Speech:** speech to text, text to speech, speech translation, speech as an agent modality.
- **Image and video generation:** text-to-image, inpainting and mask edits, video generation and editing.
- **Translator and Language features** beyond what the handler summary uses.
- **Private networking:** private endpoints, VNet integration, network isolation for Foundry and AI Search.
- **Multi-agent orchestration:** connected agents and workflows. Optional stretch goal: split intake and status into two agents.

Repository layout

FOLDER	CONTENTS
`infra/`	Bicep: Foundry, deployments, AI Search, Storage, App Insights, role assignments
`src/ClaimsAssistant.Api`	ASP.NET Core: chat endpoint, function-tool handlers, uploads, handler page
`src/ClaimsApi.Mock`	Read-only mock Claims API with SQLite seed data
`data/`	Policy documents, claim forms, selected photos, `claims.db`; `data/scripts/` holds the Python loading scripts
`eval/`	Python: test set (JSONL) and evaluation script
`notebooks/`	Python study notebooks for the studied-not-built topics